jigsaw-agile-community-rules

[8位] Qwen3-14B*3 + Llama2-13B*1 + bge-base-en-v1.5

Rank	8
Team	Reddit comment moderators
Author	muyouqian4
Topic ID	613195
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/jigsaw-agile-community-rules/discussion/613195
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

わあ！ コンペのホストである Jigsaw/Conversation AI と主催者の皆さんに、このコンペを開いていただいたことを心から感謝します。参加する前、私は NLP についてほとんど何も知りませんでしたが、このコンペから非常に多くの貴重な教訓を学びました。

素晴らしい議論の雰囲気を持つ Kaggle コミュニティと、自分の成果を惜しみなくオープンソースにしてくれる Kaggler の皆さんに大きな感謝を。

チームメイトの @williamwu88、@justforfun44 の貴重な提案と、経験と努力の共有にも大きな感謝を！ 次回もまた楽しく協力できるのを楽しみにしています！

以下の内容はすべて、テスト段階で行った学習と推論について説明しています。

完全なルールとデータにアクセスできない中で、オンライン学習は Jigsaw コミュニティの共通認識になりました。そのため私たちは、与えられた時間制限とメモリ制約の中で、より大規模で、より多くの LLM モデルをどうアンサンブルするかが鍵だと考えました。

Kaggle の「過酷な」環境で、unsloth（https://huggingface.co/unsloth）に基づき、T4*2 上で12時間以内に、3つの Qwen3-14B、1つの Llama2-13B、2つの Llama3.1-8B のモデルの効率的な学習と推論（QLora）を実現しました。CV はなく、コードは LB だけに基づいて調整しました。各提出は、同じモデルとパラメータを4つの異なるシードで学習したもののアンサンブルで、ノイズではなく安定した改善であることを確かめました。LB はランダムな分割です。55k の 29% のデータ量は小さくありません。mAUC の評価指標はかなり安定しているので、LB は信頼できると考えました。

これは、2つの最終提出のモデル構成です。

LB=0.933,PB=0.927

seed	model
42	qwen3-14b
43	qwen3-14b
44	qwen3-14b
45	llama2-13b
46	bge-base-en-v1.5

LB=0.932,PB=0.926

seed	model
42	qwen3-14b
43	qwen3-14b

seed	model
44	qwen3-14b
45	llama2-13b
46	llama3.1-8b
47	llama3.1-8b

1. 分類ヘッドの追加:

- Unsloth の公式のテキスト分類の実装は SFT によるものです（https://colab.research.google.com/github/timothelaborie/text_classification_scripts/blob/main/unsloth_classification.ipynb）。私たちはそれを標準的な AutoModelForSequenceClassification のヘッドに変更しました。単にその方が一般的だからです。どちらが良いかは分かりません。

2. 並列学習、T4*2 をフル活用する:

unsloth の並列学習の公式コードが見つからなかったので、標準的な並列学習のコードを修正・適応しました。いくつかの落とし穴⍰がありました。これを他のタスクに適応したい場合は、次の点に注意してください。

- QWEN3-14B モデルは2つのモデルを並列に学習することしかできず、8B モデルのような batch*2 の学習は使えません。8B モデルにはそのような制限はありません。おそらく、モデルはまず GPU:0 に同時にロードされる必要があり、OOM にならなかった場合にのみ2枚の GPU に分配されるためだと推測しています。この問題は解決できませんでした。Llama2-13B やそれより小さいモデルは並列に学習できます。
- Unsloth は学習の前に初期化する必要があります。

3. 学習パラメータの最適化:

主に group_by_length=True で、学習が2倍速くなりました。

4. データセット

- 各 (body, rule) の組み合わせについて、最も頻度の高いラベルを見つけ、サンプルを重複除去する。
- 公式のラベルなしデータセットを見つけた。ルールの positive サンプルの基準は分からなかったが、絶対的な negative サンプルは特定でき、それを学習セットに加えた。
- 公式のラベルなしデータセットを見つけた。ルールに基づいて推測した positive サンプルを学習セットに加えることを試した。LB が下がった場合は元に戻した。

5. 推論

- データを2枚の GPU に均等に分配し、短いものから長いものへソートして、並列に推論する。これで4倍速くなった。
- 注意: 衝突を避けるため、GPU:1 は数秒待つ必要がある。
- データを3つのバケットに分け、それぞれに異なる batch_size を指定して推論した。推論時間が減り、8B モデルをもう1つ加えられた。

6. アンサンブル

- グローバルな順位に基づくアンサンブルは、ルール内の順位に基づくアンサンブルより良かった。

7. 私たちにとってうまくいかなかったこと

- 疑似ラベル付け
- 学習中に CV をシミュレートすること

8. 他の Kaggler の皆さんの惜しみない共有を楽しみにしています！

（以下、原文には同じ内容の中国語版が続きます。その日本語訳を示します。）

わあ！ コンペのホストである Jigsaw/Conversation AI と主催者の皆さん、コンペを開催していただき本当にありがとうございます。参加する前、私は NLP についてほとんど何も知りませんでしたが、そこから多くの貴重な経験を学びました。

Kaggle の良い議論コミュニティの雰囲気と、惜しみなくオープンソースにしてくれる Kaggler の皆さんに本当に感謝します。

チームメイトの @williamwu88、@justforfun44 の valuable suggestion and for sharing experience and effort に本当に感謝します！ 次回も楽しく協力できるのを楽しみにしています！

以下の内容はすべて、テスト時に行った学習と推論です。

すべてのルールとデータを得られない状況で、オンライン学習が Jigsaw コミュニティの共通認識になりました。そのため私たちは、規定の時間制限と VRAM の制限の下で、より大規模で、より多くの LLM モデルをどうアンサンブルするかが鍵だと考えました。

Kaggle の「過酷な」環境の下で、12時間以内に T4*2 上で unsloth（https://huggingface.co/unsloth）に基づく、3つの Qwen3-14B、1つの Llama2-13B、2つの Llama3.1-8B の効率的な学習と推論（QLora）を実現しました。CV はなく、コードは LB だけに基づいて調整し、各提出は4つの異なるシードによる同じモデルとパラメータ

のアンサンブルで、ノイズではなく安定した改善であることを確かめました。LB はランダムな分割で、55k の 29% のデータ量は小さくなく、mAUC の評価指標は比較的安定しているので、LB は信じられると考えました。

これは、最終提出の2つのコードのモデル構成です:

LB=0.933,PB=0.927

seed	model
42	qwen3-14b
43	qwen3-14b
44	qwen3-14b
45	llama2-13b
46	bge-base-en-v1.5

LB=0.932,PB=0.926

seed	model
42	qwen3-14b
43	qwen3-14b
44	qwen3-14b
45	llama2-13b
46	llama3.1-8b
47	llama3.1-8b

1. 分類ヘッドの追加:

1. Unsloth の公式のテキスト分類の実装は SFT によるもの（https://colab.research.google.com/github/timothelaborie/text_classification_scripts/blob/main/unsloth_classification.ipynb）で、私たちは標準的な AutoModelForSequenceClassification の分類ヘッドに変更しました。単に慣習に合うからで、どちらが良いかは分かりません。

2. 並列学習、T4*2 をフル活用する:

unsloth の公式の並列学習に関するコードが見つからなかったので、通常の並列学習のコードをもとに修正・適応しました。いくつか落とし穴[?]があるので、他のタスクに変更したい場合は注意してください:

1. QWEN3-14B モデルは2つのモデルを並列に学習することしかできず、8B モデルのような batch*2 の学習は使えませんが、8B モデルには制限がありません。モデルはまず同時に GPU:0 にロードされる必要があり、OOM がなければその後2枚の GPU に分配されるのだと推測しています。私たちはこの問題を解決できませんでした。llama2-13B やそれより小さいモデルは並列に学習できます。
2. Unsloth の初期化は学習の前に行う必要があります。

3. 学習パラメータの最適化:

主に group_by_length=True で、2倍速くなりました。

4. データセット

1. 各 (body, rule) の組み合わせの中で最も多く現れるラベルを見つけ、サンプルを重複除去する
2. 公式のラベルなしデータセットを見つけた。ルールの正例の基準は知りようがないが、絶対的な負例は見つけられるので、学習セットに加えた
3. 公式のラベルなしデータセットを見つけ、ルールに基づいて推測した正例を学習セットに加えることを試し、LB が下がったら元に戻した

5. 推論

1. データを2枚の GPU に均等に分配し、短いものから長いものへデータをソートし、並列に推論して、4倍速くなった
2. 衝突を避けるため、GPU:1 は数秒待つ必要があることに注意
3. データを3つのバケットに分け、それぞれ batch_size を指定して推論し、推論時間を減らして、8B モデルをもう1つ加えられるようにした

6. アンサンブル

1. グローバルな順位のアンサンブルは、ルール内の順位のアンサンブルより良かった

7. 私たちにとってうまくいかなかったこと

1. 疑似ラベル
2. 学習時に CV をシミュレートすること

8. 他の Kaggler の皆さんの惜しみない共有を楽しみにしています！